Microsoft Teams Meetings, Webinars & Policy Configuration

Implementation Solution Guide

Version 1.0.0 · 2026-07-07

Contents

Executive Summary 5

Outcome . . . 5

Approach . . . 5

Why it is low-risk . . . 5

Boundary . . . 5

1. Solution Overview — Microsoft Teams Meetings, Webinars & Policy Configuration 6

What this solution is . . . 6

Scope — what this is, and what it deliberately is NOT . . . 6

Why this matters . . . 7

How it works . . . 7

The reference model (artifacts this solution creates) . . . 8

Benefits . . . 9

Limitations (be honest with buyers) . . . 9

The standard deployment model . . . 9

2. Requirements 10

2.1 Teams Premium licensing — required only for templates, sensitivity, watermark (verify per organizer) . . . 10

2.2 Least-privilege admin roles for meeting policy work . . . 10

2.3 Cohort groups for policy assignment (source of truth confirmed) . . . 10

2.4 A defined meeting/security baseline (what the org actually wants) . . . 11

2.5 Recording storage understood (OneDrive / SharePoint) . . . 11

2.6 Sensitivity label(s) defined by compliance (if using templates) . . . 12

2.7 Commercial vs Government cloud confirmed . . . 12

Global Requirements — Microsoft Teams Solutions 12

1. Microsoft 365 tenant with Microsoft Teams and appropriate licensing . . . 13

2. Administrative access (least-privilege Teams roles) . . . 13

3. Microsoft Teams PowerShell + Graph toolchain . . . 13

4. Identity, groups & lifecycle source of truth . . . 13

5. A pilot scope and change control . . . 14

6. Security, compliance & governance stakeholders . . . 14

3. Implementation 14

3.0 Assess — capture the current posture (read-only) . . . 14

3.1 Design — decide the policy set (no changes yet) . . . 15

3.2 Configure — org-wide meeting settings (`Meeting Config – Org – Prod`) . . . 16

3.3 Configure — build the meeting policies (safe, idempotent) . . . 16

3.4 Configure — webinars & town halls (`Events Policy – Standard – Prod`) . . . 16
3.5 Configure — meeting templates + sensitivity (Teams Premium — flag & verify) . . . 17
3.6 Pilot — assign policies to the pilot group only . . . 17
3.7 Validate the pilot (see §5 for full procedures) . . . 18
3.8 Expand — assign production cohorts in waves . . . 18
3.9 Confirm recording storage & expiration . . . 18
3.10 Document & hand over . . . 19
3.11 Rollback (have this ready before you start) . . . 19
5. Testing & Validation 19
Test 1 — Named policies exist and match design . . . 19
Test 2 — Org anonymous-join stance applied . . . 20
Test 3 — Group assignments present and ranked . . . 20
Test 4 — Effective policy per user is correct . . . 20
Test 5 — Lobby behaves as configured (live) . . . 20
Test 6 — Recording & transcription follow policy (live) . . . 21
Test 7 — Presenter role enforced (live) . . . 21
Test 8 — Webinars & town halls governed . . . 21
Test 9 — Meeting template + sensitivity (Teams Premium only) . . . 21
Test 10 — Recording lands in the right place with expiration . . . 21
Test 11 — No unintended blast radius . . . 22
6. Troubleshooting 22
A policy change "isn't taking effect" for users . . . 22
A user gets the wrong policy (multiple groups) . . . 23
Anonymous users still join when you disabled it (or can't when you allowed it) . . . 23
Lobby lets in people it shouldn't (or holds everyone) . . . 23
Presenter role isn't enforced — attendees can still share . . . 23
Recording or transcription button is missing (or unexpectedly present) . . . 23
Recording succeeds but the file is missing / in the wrong place . . . 24
Webinar or town hall creation is blocked (or wrongly allowed) . . . 24
`Set-CsTeamsEventsPolicy` / template cmdlet fails on a parameter . . . 24
Meeting template doesn't appear for a user . . . 24
Sensitivity label can't be attached to the template . . . 24
`Connect-MicrosoftTeams` or Graph fails / wrong cloud . . . 25
Rolling back safely . . . 25
7. Best Practices 25
Policy design . . . 25

Security defaults that pay off . . . 26
Recording, transcription & storage . . . 26
Webinars & town halls . . . 26
Teams Premium (templates & sensitivity) . . . 26
Assignment & change discipline . . . 27
Cloud & versioning . . . 27
8. Deliverables Summary 27
Documentation . . . 27
PowerShell scripts (5) — MicrosoftTeams module / Microsoft Graph . . . 28
Appendix A — PowerShell Scripts 30

Executive Summary

Outcome

Teams meetings stop behaving inconsistently and permissively by default and become **secure, consistent, and fit-for-purpose by policy**. Concretely: the **lobby**, **anonymous and PSTN join**, **cloud recording and transcription**, **presenter roles**, and **recording expiration/storage** are set by named meeting policies per cohort; **webinars and town halls** are governed so only the intended creators can run external events; sensitive meetings get a **template and sensitivity label** (forced lobby, no recording, watermark) where **Teams Premium** is licensed; and every policy is **assigned by group**, so the right people automatically get the right experience.

Approach

This is Teams **meeting and webinar policy configuration** implemented specifically — not a broad Teams governance or voice project. It follows the house model **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand**: today's posture is assessed, a small named policy set is designed with the business (`Standard` / `Exec` / `Restricted` plus a tuned Global baseline), org and policy settings are configured, everything is assigned to a small **pilot group** first, validated against explicit pass/fail gates (lobby, recording, presenter, webinars, template all correct), and only then expanded to production cohorts in controlled waves. Automation is delivered as five **MicrosoftTeams** / Microsoft Graph scripts that are read-only or `-WhatIf` by default, with no stored credentials.

Why it is low-risk

- **Nothing production changes until it's assigned.** Policies are built and validated while assigned to no one; a custom policy left unassigned affects zero users.
- **Pilot-first with gates and a fast rollback.** Every change is proven on a pilot group before broad rollout, and rollback is clean — un-assign the group (users fall back to the previous/Global policy), no data touched.
- **Licensing is verified up front.** Templates, sensitivity, and watermark are **Teams Premium** — the solution checks entitlement and flags it rather than assuming it.
- **Compliance stays in control of compliance.** The solution *attaches* the sensitivity label the compliance owner defined in Purview; **it never decides what the label protects or how long meeting data is retained.**

Boundary

Scoped to **meeting policies, webinars/town halls, templates + sensitivity attachment, recording storage/expiration, and group-based assignment**. **Teams Phone / calling**, the **definition** of sensitivity/retention labels, and broader **Teams governance / guest access / apps / Rooms** are **separate products** and intentionally out of scope here.

1. Solution Overview — Microsoft Teams Meetings, Webinars & Policy Configuration

What this solution is

Out of the box, Teams meetings behave the same for everyone, and the defaults are permissive: anyone can bypass the lobby, external and anonymous people can join and even start meetings, everyone is a presenter, cloud recording and transcription are on, and there is no way to tell an exec's board meeting apart from a casual team sync. This solution **configures Microsoft Teams meetings, webinars, and town halls by policy** so meetings are secure, consistent, and fit-for-purpose. Specifically it:

• designs and applies **Teams meeting policies** that control the **lobby** (who bypasses / who waits), **anonymous and PSTN join**, **cloud recording and transcription**, **presenter roles**, meeting chat, and **recording expiration / storage**;

• configures **webinars and town halls** through the **Teams events policy** (who can create webinars, who can run town halls, attendee access), and notes where legacy **Teams live events** still apply;

• builds **meeting templates** and attaches **sensitivity labels** to meetings where **Teams Premium** is licensed (watermark, prevent copy, forced lobby/recording) — flagging the licensing gate;

• confirms **recording storage** lands correctly in **OneDrive / SharePoint** and sets recording **expiration**; and

• **assigns policies by group** (Entra ID / Microsoft 365 group) with `New-CsGroupPolicyAssignment`, so the right cohort gets the right meeting experience without per-user churn.

The work is done in the **Teams admin center** (admin.teams.microsoft.com), the **Microsoft 365 admin center**, the **Microsoft Purview** portal (for sensitivity labels), and the **MicrosoftTeams PowerShell module** (with **Microsoft Graph** for group membership). Every change is **pilot-first**.

Scope — what this is, and what it deliberately is NOT

In scope: Teams **meeting policies**, org-wide **meeting settings** (anonymous join, storage), **events policy** (webinars + town halls), **meeting templates** and their **sensitivity label** attachment (Teams Premium), **recording expiration/storage** behavior, and **group-based policy assignment**.

Not in scope (separate products / owners):

• **Teams Phone / calling plans / Operator Connect / audio conferencing dial-in** — voice is a different solution. This solution touches PSTN only as a *meeting lobby/bypass* control, not as call routing or number provisioning.

• **The definition of sensitivity labels and retention** — a meeting template can *apply* a sensitivity label, but **what the label means, and any retention/records schedule, is a Microsoft Purview / compliance determination** made by the compliance/records/legal owners. This solution attaches the label the compliance owner names; it does not author label policy.

• **Broad Teams governance / lifecycle** (team creation, naming, expiration), **external & guest access **governance, and** Teams apps** governance are their own solutions.

• **Teams Rooms** device configuration — a Rooms meeting experience keys off these same meeting policies, but device deployment is a separate solution.

Why this matters

• **The default lobby is a security gap.** With permissive defaults, uninvited external or anonymous participants can land directly in a meeting, and anyone can start it before the organizer arrives. A lobby policy closes that gap consistently instead of meeting-by-meeting.

• **"Everyone is a presenter" is how meetings get hijacked.** Setting the **presenter role** by policy stops attendees from sharing their screen or removing others uninvited.

• **Recording and transcription are compliance-sensitive.** Whether a meeting can be recorded or transcribed, where the recording is stored, and how long it lives are decisions that should be set by policy for a cohort — not left to each user.

• **Webinars and town halls are external-facing.** Uncontrolled, anyone can spin up an external webinar. The events policy decides who can, and how attendees join.

• **One size does not fit all.** Execs, standard staff, and frontline/external-facing users need different meeting experiences. Group-based assignment delivers that without manual per-user work.

How it works

1. **Licensing gates the premium pieces (verify first).** Core meeting policies, events policy, and group assignment work on standard Teams licensing. **Meeting templates, sensitivity labels on meetings, watermark, and "prevent copy" are Teams Premium features** — they require a **Teams Premium** license on the organizer. Confirm entitlement before designing templates. *(Verify in your tenant/plan.)*

1. **Assess the current posture.** Export today's meeting policies, events policy, org meeting configuration, recording/storage settings, and existing group assignments — the `Meeting Posture – Teams – Discovery` snapshot — so you design against reality, not assumptions.

1. **Design the policy set.** Rather than editing the built-in **Global (Org-wide default)** policy destructively, design a small set of **named custom policies** for the cohorts that need them, and tune the Global policy to a safe baseline for everyone else.

1. **Configure org-wide meeting settings.** Set the org-level `Set-CsTeamsMeetingConfiguration` controls — notably `-DisableAnonymousJoin` — which sit *above* the per-policy anonymous toggle.

1. **Build the meeting policies.** Create/tune each policy's lobby (`-AutoAdmittedUsers`), anonymous join (`-AllowAnonymousUsersToJoinMeeting`), PSTN lobby bypass (`-AllowPSTNUsersToBypassLobby`), recording (`-AllowCloudRecording`), transcription

(`-AllowTranscription`), presenter role (`-DesignatedPresenterRoleMode`), chat, and recording expiration (`-NewMeetingRecordingExpirationDays`).

1. **Configure webinars & town halls.** Create/tune the **events policy**

(`Set-CsTeamsEventsPolicy`): `-AllowWebinars`, `-AllowTownhalls`, attendee access, and event integrations. Note the legacy **Teams live events** policy (`Set-CsTeamsMeetingBroadcastPolicy`) where an org still uses live events; Microsoft is steering live events toward **town halls**.

1. **Build templates + sensitivity (Teams Premium).** In the Teams admin center, create the meeting

template and attach the **sensitivity label** the compliance owner specifies, then publish it with a template policy so the right cohort sees it.

1. **Assign by group.** Assign each policy to its cohort's group with

`New-CsGroupPolicyAssignment` (ranked), so membership drives the meeting experience — pilot group first.

The reference model (artifacts this solution creates)

Every artifact is named `<Purpose> – <Scope> – <Stage>` and reused verbatim across the docs, scripts, the Teams admin center, and the Purview portal.

Artifact	Name
Read-only posture assessment export	`Meeting Posture – Teams – Discovery`
Org-wide meeting settings (anonymous join, storage)	`Meeting Config – Org – Prod`
Baseline meeting policy for most staff	`Meeting Policy – Standard – Prod`
Executive / sensitive-meeting policy	`Meeting Policy – Exec – Prod`
Restricted (frontline / external-facing) policy	`Meeting Policy – Restricted – Prod`
Pilot copy of the standard policy	`Meeting Policy – Standard – Pilot`
Webinars + town halls events policy	`Events Policy – Standard – Prod`
Restricted events policy (no external webinars)	`Events Policy – Restricted – Prod`
Meeting template (Teams Premium)	`Meeting Template – Confidential – Prod`
Sensitivity label attached to the template	`Sensitivity Label – Confidential – Meeting` *(defined by compliance)*
Pilot scope group	`Meeting Scope – Pilot – Group`
Executive scope group	`Meeting Scope – Exec – Group`
Group policy assignment (exec cohort)	`Meeting Assignment – Exec – Group`
Validation pass/fail report	`Meeting Validation – Teams – Report`

> *The **sensitivity label** name above is illustrative — use the exact label the **compliance/records** owner has defined in Purview. Never invent a label or decide what it protects.*

Benefits

• Lobby, recording, transcription, and presenter behavior are **set by policy**, not left to defaults — consistently, per cohort.

• Anonymous and PSTN join are controlled at **both** the org level and the policy level, closing the most common meeting-security gaps.

• Webinars and town halls are **governed** — only the intended creators can run external events.

• Sensitive meetings get a **template + sensitivity label** (watermark, prevent copy, forced lobby) where Teams Premium is licensed.

• Assignment is **group-driven**, so onboarding a new exec or team gets the right experience automatically, and every change is **pilot-first with pass/fail gates and a rollback**.

Limitations (be honest with buyers)

• **Templates, sensitivity labels on meetings, watermark, and prevent-copy are Teams Premium.** Without Premium on the organizer, those steps do not apply — **verify entitlement in your tenant/plan**; this solution flags it but cannot grant it.

• **Policy changes are not instant.** New or changed meeting/events policies and group assignments can take time to propagate to clients (commonly minutes, occasionally longer); validate after propagation.

• **Group assignment is ranked, not additive.** A user in multiple assigned groups gets the policy from the **highest-ranked** assignment for that policy type — plan ranks deliberately (covered in Implementation).

• **The Global policy still applies to everyone unassigned.** Tuning custom policies does nothing for users who fall back to Global — the baseline must be set too.

• **Sensitivity-label meaning and retention are compliance decisions.** This solution attaches the label the compliance owner specifies; **what it protects and how long meeting data is retained is their determination**, not the implementer's.

• **Commercial vs Government clouds differ.** Portal URLs and some capability availability differ on **GCC / GCC High / DoD** (`*.office365.us`, `*.microsoft.us`); noted where it matters — verify on your cloud.

The standard deployment model

> *Assess the current meeting/events posture and licensing → design the policy set (Global baseline + Standard/Exec/Restricted, events policy, templates) → configure org settings and policies → assign to a **pilot** group first → validate (lobby, recording, roles, webinars, template) against explicit pass/fail gates → expand to the broad cohorts in waves. Pattern: **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand**, with explicit pass/fail gates and a documented rollback.*

2. Requirements

This solution assumes the **shared Teams category requirements** are already met — a Microsoft 365 tenant with Teams and appropriate licensing, least-privilege **Teams Administrator** access, the **MicrosoftTeams** PowerShell + **Microsoft Graph** toolchain, an identity/group source of truth, a pilot scope with change control, and engagement with security/compliance/governance stakeholders. See `global-requirements-teams.md` for those six; do **not** duplicate them. Below are the prerequisites **specific to meetings, webinars, templates, and policy assignment**.

2.1 Teams Premium licensing — required only for templates, sensitivity, watermark (verify per organizer)

• **Description:** Core meeting policies, the events policy, org meeting settings, and group assignment work on standard Teams licensing. But **meeting templates, applying a sensitivity label to a meeting, watermark, "prevent copy," and advanced meeting protection are Teams Premium features** — the **organizer** of the meeting must hold a **Teams Premium** license for them to take effect.

• **Why required:** If you design the template/sensitivity portion of this solution but the target cohort has no Teams Premium, those steps silently do nothing while the core policies still apply. This is the single most common surprise.

• **How to verify:** In the Microsoft 365 admin center, check **Billing > Licenses** for **Teams Premium **and which users hold it. In PowerShell you can confirm feature availability by inspecting whether template/sensitivity cmdlets and admin-center options are present.** Treat entitlement as "verify in your tenant/plan" — do not assume.**

• **How to implement if missing:** Either assign Teams Premium to the cohort that needs template/sensitivity features, or scope this deployment to the **core policy** work only (lobby, recording, roles, webinars, assignment) and defer templates until Premium is available.

2.2 Least-privilege admin roles for meeting policy work

• **Description:** The work needs the **Teams Administrator** role for meeting/events policy and group assignment. Creating or editing **sensitivity labels** requires a **Compliance/Records** role in **Microsoft Purview** (**Sensitivity Label Administrator** / Compliance Administrator) — held by the compliance owner, not necessarily the Teams admin. **Global Administrator** is not required for the core work.

• **Why required:** Meeting and events policies are gated by the Teams Administrator role; sensitivity labels are gated by the Purview compliance roles. Splitting them keeps least privilege intact.

• **How to verify:** In the M365 admin center / Entra roles confirm **Teams Administrator** on the operator; confirm the compliance owner holds the Purview label role.

• **How to implement if missing:** Have a Global Admin assign **Teams Administrator** to the operator under change control; route label creation to the compliance owner.

2.3 Cohort groups for policy assignment (source of truth confirmed)

• **Description:** Group-based assignment needs the **Entra ID / Microsoft 365 security groups** that represent each meeting cohort — at minimum a **pilot** group and the production cohorts (e.g. exec, standard, restricted). Group policy assignment supports security groups and M365 groups; **very large or dynamic groups** may evaluate more slowly — verify behavior in your tenant.

• **Why required:** `New-CsGroupPolicyAssignment` targets a group, not individual users. Without clean, authoritative cohort groups, assignment is guesswork and drifts from HR/identity reality.

• **How to verify:** Confirm the group object IDs (`Get-MgGroup`), their membership source (assigned vs dynamic), and that they are the authoritative cohorts. Confirm the pilot group exists and is small.

• **How to implement if missing:** Create or nominate the cohort groups (naming per the reference model, e.g. `Meeting Scope – Exec – Group`) before assigning policies.

2.4 A defined meeting/security baseline (what the org actually wants)

• **Description:** The desired settings for each cohort must be **decided before configuring**: lobby behavior (who bypasses), whether anonymous/PSTN can join and start, whether cloud recording and transcription are allowed, the default presenter role, meeting-chat behavior, and recording expiration.

• **Why required:** These are policy decisions, not technical ones — the tool applies them, but the security/collaboration owners decide them. Configuring without an agreed baseline produces churn.

• **How to verify:** Confirm a signed-off baseline table (per cohort) exists, reviewed by the security and collaboration-governance stakeholders (GR §6).

• **How to implement if missing:** Facilitate a short baseline decision with the stakeholders using the `Meeting Posture – Teams – Discovery` export as the starting point.

2.5 Recording storage understood (OneDrive / SharePoint)

• **Description:** Teams meeting recordings are stored in **OneDrive** (for non-channel meetings, in the organizer's OneDrive) or **SharePoint** (for channel meetings, in the team's site) — **not** in a separate recordings service. Recording behavior therefore depends on the organizer having a provisioned OneDrive and sufficient storage, and on OneDrive/SharePoint retention.

• **Why required:** If an organizer has no OneDrive license/provisioning, recording can fail or have nowhere to land. Recording **expiration** (`-NewMeetingRecordingExpirationDays`) and any OneDrive/SharePoint retention interact.

• **How to verify:** Confirm in-scope organizers have OneDrive provisioned and adequate quota; note any OneDrive/SharePoint retention labels that affect recordings (a compliance concern — coordinate).

• **How to implement if missing:** Ensure OneDrive is licensed/provisioned for organizers who record; coordinate recording expiration with the compliance owner's retention stance.

2.6 Sensitivity label(s) defined by compliance (if using templates)

• **Description:** If the deployment attaches a **sensitivity label** to a meeting template, the label must already be **defined and published in Microsoft Purview** by the compliance owner, with its meeting-protection settings (encryption, watermark, prevent copy, forced lobby/recording) decided.

• **Why required:** A template can only attach a label that exists; **what the label protects is a compliance determination**, deferred to that owner.

• **How to verify:** Confirm the exact label name(s) and that they are published to the relevant users.

• **How to implement if missing:** Request the compliance owner define/publish the label first; do not invent a label or its protections.

2.7 Commercial vs Government cloud confirmed

• **Description:** Confirm whether the tenant is **Commercial** or **Government** (**GCC / GCC High / DoD**). Admin-center URLs, some capability availability (including certain Premium and events features), and the PowerShell endpoint differ by cloud.

• **Why required:** Steps and URLs that are correct on Commercial may differ on Government clouds (`*.office365.us`, `*.microsoft.us`); assuming the wrong cloud produces broken links and missing features.

• **How to verify:** Confirm the cloud with the tenant owner; when connecting PowerShell, use the correct environment (e.g. `Connect-MicrosoftTeams -TeamsEnvironmentName TeamsGCCH` / `TeamsDOD` where applicable — **verify the exact parameter value for your cloud**).

• **How to implement if missing:** Record the cloud in the runbook and use the matching URLs/endpoints throughout.

Global Requirements — Microsoft Teams Solutions

These requirements apply to **every** solution in the **Teams** category (Teams governance, meetings, external access, Teams Phone, apps, Teams Rooms, and Teams security/compliance). Verify each before starting a solution's implementation steps.

> ***Vendor boundary (applies to every Teams solution).*** *The **Teams admin center** (admin.teams.microsoft.com), the **Microsoft 365 admin center**, the **MicrosoftTeams PowerShell module**, and **Microsoft Graph** are authored at exact, current click-by-click / cmdlet detail from Microsoft's public documentation. Where a capability, policy, or portal path is **plan- or version-dependent** (Teams Phone licensing, Calling Plans vs Operator Connect vs Direct Routing, Premium features, Commercial vs Government cloud), it is **cited or flagged "verify in your tenant"** — never invented. **Third-party devices and apps** (certified Teams Rooms hardware, SBCs, ISV apps) are integrated at the pattern level; their internals are deferred to the vendor's own docs. Automation is **PowerShell** (MicrosoftTeams module / Graph) — never with hardcoded credentials or*

secrets.

1. Microsoft 365 tenant with Microsoft Teams and appropriate licensing

• **Description:** A Microsoft 365 tenant with **Microsoft Teams** enabled and the licensing the capability requires (e.g. Teams Phone + a calling plan/Operator Connect for voice, Teams Premium for premium features, the appropriate E-plan for advanced compliance).

• **Why required:** All policy, voice, app, and compliance configuration depends on the right plan.

• **How to verify:** In the M365 admin center confirm the Teams subscription and relevant add-ons.

• **How to implement if missing:** Assign/purchase the required licensing before starting.

2. Administrative access (least-privilege Teams roles)

• **Description:** Access with the least-privilege role the solution names — typically **Teams Administrator **(or Teams Communications/Device Administrator), with** Global Administrator** only where genuinely required.

• **Why required:** Teams policy, voice, app, and org-wide settings require the corresponding role.

• **How to verify:** Confirm the role in the M365 admin center / Entra roles.

• **How to implement if missing:** Have a Global Admin assign the least-privilege role under change control.

3. Microsoft Teams PowerShell + Graph toolchain

• **Description:** A workstation with the **MicrosoftTeams** PowerShell module (and Graph PowerShell where used), modern-auth / MFA-capable admin sign-in.

• **Why required:** Configuration, automation, and the provided scripts run through Teams PowerShell / Graph.

• **How to verify:** `Connect-MicrosoftTeams` succeeds; `Get-Module MicrosoftTeams`.

• **How to implement if missing:** `Install-Module MicrosoftTeams`; confirm modern-auth sign-in.

4. Identity, groups & lifecycle source of truth

• **Description:** A defined identity/group source (Entra ID groups, and where used **Microsoft 365 Groups **which back teams) and the** lifecycle** stance (creation, naming, expiration/retention) the org expects.

• **Why required:** Teams are backed by M365 Groups; governance, provisioning, and access key off groups and the lifecycle model.

• **How to verify:** Confirm which groups/OU model is authoritative and the lifecycle expectations.

• **How to implement if missing:** Define the group/lifecycle model before configuring governance.

5. A pilot scope and change control

• **Description:** A **pilot team / policy group / user set** to validate on before org-wide rollout,

and a **change-control** process for production policy/voice/compliance changes.

• **Why required:** Every solution follows an **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand**

model; policy and voice changes are high-impact.

• **How to verify:** Confirm a named pilot scope and an approval/change window are available.

• **How to implement if missing:** Create a pilot scope and agree change control before production changes.

6. Security, compliance & governance stakeholders

• **Description:** Engagement with the **security, compliance/records, and collaboration-governance**

stakeholders whose policies the configuration must satisfy (external access, information barriers, retention, DLP, app governance).

• **Why required:** Collaboration governance and compliance settings must align to enterprise policy.

• **How to verify:** Confirm the applicable policies and named approvers.

• **How to implement if missing:** Identify the owners and capture requirements before configuring governance.

*The deployment model across every Teams solution is the same: **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand**, with explicit pass/fail gates and a documented rollback for production changes. Automation is delivered as **PowerShell** (MicrosoftTeams module / Graph) — never with hardcoded credentials or secrets in code.*

3. Implementation

Follow these steps in order. They implement the house model **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand**. Every artifact is named per the reference model and reused verbatim. Meeting, events, and assignment changes are **high-impact** (they change every affected user's meeting experience) — do them under change control, **pilot-first**, with the rollback in §3.11 ready.

> ***Connect once, correctly.*** *All PowerShell below uses modern auth — no stored credentials: `powershell Connect-MicrosoftTeams # Commercial. For Government, add -TeamsEnvironmentName # (e.g. TeamsGCCH / TeamsDOD) — verify the value for your cloud. Connect-MgGraph -Scopes 'Group.Read.All','User.Read.All' # for group membership / effective policy ` Admin center:* ***admin.teams.microsoft.com*** *(Government: verify the* `*.office365.us` *equivalent).*

3.0 Assess — capture the current posture (read-only)

1. Run the discovery script to snapshot today's state into `Meeting Posture – Teams – Discovery`:

```
.\TEAMSMTG-Discovery.ps1 -CsvFolder .\posture
```

It enumerates every **meeting policy** (`Get-CsTeamsMeetingPolicy`), the **events policy** (`Get-CsTeamsEventsPolicy`), the org **meeting configuration** (`Get-CsTeamsMeetingConfiguration`), recording/storage-relevant settings, and existing **group policy assignments** (`Get-CsGroupPolicyAssignment`).

1. **Gate G0 (baseline captured):** you have a CSV of current policies and assignments, and the security/collaboration owners have signed off the **target baseline** per §2.4. If not, stop and get the baseline first — do not configure against assumptions.

3.1 Design — decide the policy set (no changes yet)

Design a **small** set rather than editing Global destructively:

Cohort	Policy name	Lobby (`-AutoAdmittedUsers`)	Anonymous join	Recording	Presenter role
Most staff	`Meeting Policy – Standard – Prod`	`EveryoneInCompanyExcludingGuests`	Off	On	`EveryoneUserOverride`
Execs / sensitive	`Meeting Policy – Exec – Prod`	`OrganizerOnly` (or `InvitedUsers`)	Off	Off (or On per policy)	`OrganizerOnlyUserOverride`
Frontline / external-facing	`Meeting Policy – Restricted – Prod`	`EveryoneInCompany`	Off	Off	`EveryoneInCompanyUserOverride`
Everyone unassigned	**Global (Org-wide default)**	`EveryoneInCompanyExcludingGuests`	Off	per org baseline	`EveryoneUserOverride`

> *The exact values are **your** baseline decisions (§2.4) — the table shows a defensible default set.* `-AutoAdmittedUsers` *accepts values including* `EveryoneInCompanyExcludingGuests`, `EveryoneInCompany`, `Everyone`, `OrganizerOnly`, `InvitedUsers`, *and* `EveryoneInSameAndFederatedCompany` *— **verify the exact accepted set for your tenant version**. Presenter-role modes include* `EveryoneUserOverride`, `EveryoneInCompanyUserOverride`, *and* `OrganizerOnlyUserOverride`.

Also design the **events policy** (`Events Policy – Standard – Prod` vs `Events Policy – Restricted – Prod`) and, if Teams Premium is licensed, the **template** (`Meeting Template – Confidential – Prod`) and its **sensitivity label** (`Sensitivity Label – Confidential – Meeting`, defined by compliance).

3.2 Configure — org-wide meeting settings (`Meeting Config – Org – Prod`)

Some controls sit **above** per-policy settings, org-wide:

1. Preview, then set the org meeting configuration — notably anonymous join:

```
# Preview
Set-CsTeamsMeetingConfiguration -Identity Global -DisableAnonymousJoin $true -WhatIf
# Apply (this is the org master switch for anonymous join)
Set-CsTeamsMeetingConfiguration -Identity Global -DisableAnonymousJoin $true
```

1. **Layered model to keep straight:** `Set-CsTeamsMeetingConfiguration -DisableAnonymousJoin` is the

org master switch; the per-policy `-AllowAnonymousUsersToJoinMeeting` (§3.3) only takes effect if the org switch permits it. Decide the org stance first.

1. **Gate G1 (org settings applied):** `Get-CsTeamsMeetingConfiguration` shows the intended

`DisableAnonymousJoin`. Anonymous join now behaves per the org decision.

3.3 Configure — build the meeting policies (safe, idempotent)

Use the configuration script in preview mode first, then commit. It creates any missing policy and updates drift on existing ones; it never deletes.

1. **Preview** the full policy set (no changes):

```
.\TEAMSMTG-Set-MeetingPolicy.ps1 -WhatIf
```

1. **Commit** the policies:

```
.\TEAMSMTG-Set-MeetingPolicy.ps1 -Commit
```

Under the hood each policy is created/tuned with real parameters, e.g. the Exec policy:

```
New-CsTeamsMeetingPolicy -Identity 'Meeting Policy – Exec – Prod' -ErrorAction SilentlyContinue
Set-CsTeamsMeetingPolicy  -Identity 'Meeting Policy – Exec – Prod' `
  -AutoAdmittedUsers 'OrganizerOnly' `
  -AllowAnonymousUsersToJoinMeeting $false `
  -AllowPSTNUsersToBypassLobby $false `
  -AllowCloudRecording $false `
  -AllowTranscription $false `
  -DesignatedPresenterRoleMode 'OrganizerOnlyUserOverride' `
  -NewMeetingRecordingExpirationDays 30
```

1. Also tune the **Global (Org-wide default)** policy to the safe baseline so unassigned users are

covered (the script does this when `-IncludeGlobal` is passed).

1. **Gate G2 (policies exist and match design):** `Get-CsTeamsMeetingPolicy | Format-Table Identity,

AutoAdmittedUsers, AllowCloudRecording, DesignatedPresenterRoleMode` shows each named policy with the designed values. No user is assigned yet — safe to proceed.

3.4 Configure — webinars & town halls (`Events Policy – Standard – Prod`)

1. Preview then create/tune the events policy:

```
.\TEAMSMTG-Set-EventsAndTemplates.ps1 -WhatIf
.\TEAMSMTG-Set-EventsAndTemplates.ps1 -Commit
```

Which applies, e.g.:

```
New-CsTeamsEventsPolicy -Identity 'Events Policy – Standard – Prod' -ErrorAction SilentlyContinue
Set-CsTeamsEventsPolicy -Identity 'Events Policy – Standard – Prod' `
  -AllowWebinars 'Enabled' -AllowTownhalls 'Enabled'
# Restricted cohort: no external webinars
Set-CsTeamsEventsPolicy -Identity 'Events Policy – Restricted – Prod' `
  -AllowWebinars 'Disabled' -AllowTownhalls 'Disabled'
```

> *Parameter names/values for the events policy are **version-dependent** (e.g. attendee-access and event-integration parameters have evolved) — the script sets the widely-available `-AllowWebinars` / `-AllowTownhalls`; **verify the exact parameters for your tenant version** and extend as needed.*

1. **Legacy live events (only if still used):** where an org still runs classic **Teams live events**,

configure `Set-CsTeamsMeetingBroadcastPolicy`. Microsoft is steering live events toward **town halls** — treat live events as legacy and prefer town halls for new work. *(Verify the retirement timeline for your cloud.)*

1. **Gate G3 (events policy correct):** `Get-CsTeamsEventsPolicy` shows the intended webinar/town-hall

stance per cohort.

3.5 Configure — meeting templates + sensitivity (Teams Premium — flag & verify)

> ***Licensing gate:** this whole step requires **Teams Premium** on the organizer (§2.1). If Premium is not licensed, **skip to §3.6** and record templates as deferred.*

1. In the **Teams admin center** → **Meetings** → **Meeting templates**, create

`Meeting Template – Confidential – Prod`. Set the template's locked options (e.g. force lobby to `OrganizerOnly`, disable recording, restrict who can present).

1. Attach the **sensitivity label** `Sensitivity Label – Confidential – Meeting` — using the exact label

the **compliance owner** published in Purview (§2.6). The label's meeting protections (watermark, prevent copy, encryption) are **the compliance owner's determination**; you attach, you do not define.

1. Publish the template to the right cohort with a **template policy** (Teams admin center; PowerShell

exposes template configuration/visibility cmdlets whose exact names are **version-dependent** — verify in your tenant, e.g. `Get-CsTeamsMeetingTemplateConfiguration` and the template policy cmdlets).

1. **Gate G4 (template visible where intended):** a pilot organizer with Premium sees

`Meeting Template – Confidential – Prod` when scheduling; a non-Premium user does not (expected).

3.6 Pilot — assign policies to the pilot group only

Do **not** assign to production cohorts yet.

1. Create the pilot copy `Meeting Policy – Standard – Pilot` (the config script created it) and assign

it to `Meeting Scope – Pilot – Group` by **group assignment**:

```
.\TEAMSMTG-AssignmentReport.ps1 -AssignGroup 'Meeting Scope – Pilot – Group' `
  -PolicyType TeamsMeetingPolicy -PolicyName 'Meeting Policy – Standard – Pilot' -Rank 1 -WhatIf
# then -Commit to apply
```

Which wraps:

```
$g = Get-MgGroup -Filter "displayName eq 'Meeting Scope – Pilot – Group'"
New-CsGroupPolicyAssignment -GroupId $g.Id -PolicyType TeamsMeetingPolicy `
  -PolicyName 'Meeting Policy – Standard – Pilot' -Rank 1
```

1. Assign the pilot **events policy** the same way (`-PolicyType TeamsEventsPolicy`).

2. **Gate G5 (pilot assigned):** `Get-CsGroupPolicyAssignment -GroupId $g.Id` lists the pilot

assignments. Allow time to propagate (minutes; occasionally longer).

3.7 Validate the pilot (see §5 for full procedures)

1. Run validation against the pilot users:

```
.\TEAMSMTG-Validation.ps1 -SampleGroup 'Meeting Scope – Pilot – Group' -CsvPath .\pilot-validation.csv
```

1. With pilot users, **schedule and start a real meeting** and confirm the lobby, recording,

transcription, and presenter behavior match the assigned policy, and (if Premium) that the template and sensitivity label appear and enforce.

1. **Gate G6 (pilot passes):** every validation row is **PASS** and the live-meeting behavior matches

design. If any fail, fix and re-pilot — **do not expand**.

3.8 Expand — assign production cohorts in waves

1. Assign each production policy to its group, ranked so multi-group users resolve deterministically

(**lower Rank = higher precedence**):

```
.\TEAMSMTG-AssignmentReport.ps1 -AssignGroup 'Meeting Scope – Exec – Group' `
  -PolicyType TeamsMeetingPolicy -PolicyName 'Meeting Policy – Exec – Prod' -Rank 1 -Commit
.\TEAMSMTG-AssignmentReport.ps1 -AssignGroup 'Meeting Scope – Standard – Group' `
  -PolicyType TeamsMeetingPolicy -PolicyName 'Meeting Policy – Standard – Prod' -Rank 2 -Commit
```

This creates the `Meeting Assignment – Exec – Group` (etc.) assignments. Assign the matching **events policies** similarly.

1. Expand in **waves** (e.g. exec cohort, then standard, then restricted), validating after each wave.

2. **Gate G7 (production assigned & effective):** `TEAMSMTG-AssignmentReport.ps1` shows the intended

effective policy per sampled user, and each wave passes §5 validation before the next.

3.9 Confirm recording storage & expiration

1. With a recording-enabled cohort, record a test meeting and confirm the recording lands in the

organizer's **OneDrive** (non-channel) or the team's **SharePoint** site (channel meeting), and that `-NewMeetingRecordingExpirationDays` is reflected on the file's expiration.

1. **Gate G8 (storage correct):** the recording is in the expected OneDrive/SharePoint location with the expected expiration. Coordinate any conflicting OneDrive/SharePoint retention with the compliance owner.

3.10 Document & hand over

Record the final policy set, events policy, template/label, group assignments and ranks, and the `Meeting Validation – Teams – Report` in the runbook (`ADMIN_DOCS`). Note the cloud (Commercial vs Government) and any Premium-gated features that were deferred.

3.11 Rollback (have this ready before you start)

Because policy/assignment changes are reversible, rollback is clean:

- **Un-assign a group policy** (revert a cohort to Global/previous):

```
Remove-CsGroupPolicyAssignment -GroupId $g.Id -PolicyType TeamsMeetingPolicy
```

Users fall back to their next-ranked assignment or the **Global** default.

- **Revert a per-user assignment** to Global: `Grant-CsTeamsMeetingPolicy -Identity <user> -PolicyName $null`.
- **Revert org anonymous stance:** `Set-CsTeamsMeetingConfiguration -Identity Global -DisableAnonymousJoin $false`.
- **A custom policy left unassigned affects no one** — you can safely leave it in place or remove it with `Remove-CsTeamsMeetingPolicy -Identity '<name>'` once no assignment references it.
- **Templates/labels:** unpublish the template policy to hide the template; the sensitivity label is owned by compliance — coordinate any change with them.

> *Roll back **assignments first** (fastest way to restore the prior experience), then policies. Never delete a policy that still has assignments.*

5. Testing & Validation

Run these after each pilot/expand wave. Each test has an explicit **pass/fail**. Most are read-only via `TEAMSMTG-Validation.ps1`; the live-meeting tests require actually scheduling and joining a meeting, which is the only way to prove the runtime experience. Reuse the reference-model names throughout.

> *Reconnect if needed:* `Connect-MicrosoftTeams` *(Government: add* `-TeamsEnvironmentName`*) and* `Connect-MgGraph -Scopes 'Group.Read.All','User.Read.All'`.

Test 1 — Named policies exist and match design

- **How:** `Get-CsTeamsMeetingPolicy | Format-Table Identity, AutoAdmittedUsers, AllowAnonymousUsersToJoinMeeting, AllowCloudRecording, AllowTranscription, DesignatedPresenterRoleMode, NewMeetingRecordingExpirationDays`

- **Pass:** `Meeting Policy – Standard – Prod`, `– Exec – Prod`, `– Restricted – Prod`, and the pilot

policy exist, each with the values from the §3.1 design table.

- **Fail:** a policy is missing or a value differs → re-run `TEAMSMTG-Set-MeetingPolicy.ps1 -Commit`.

Test 2 — Org anonymous-join stance applied

- **How:** `Get-CsTeamsMeetingConfiguration -Identity Global | Select-Object DisableAnonymousJoin`
- **Pass:** `DisableAnonymousJoin` matches the org decision (`Meeting Config – Org – Prod`).
- **Fail:** value differs → re-apply `Set-CsTeamsMeetingConfiguration` (§3.2).

Test 3 — Group assignments present and ranked

- **How:** `Get-CsGroupPolicyAssignment -PolicyType TeamsMeetingPolicy | Format-Table GroupId, PolicyName, Rank`

(resolve GroupId to name via `Get-MgGroup`).

- **Pass:** each cohort group is assigned its intended policy at the intended **Rank** (lower = higher precedence); the pilot assignment exists during pilot.
- **Fail:** missing/duplicate assignment or wrong rank → fix with the assignment helper (§3.6/§3.8).

Test 4 — Effective policy per user is correct

- **How:** `Get-CsUserPolicyAssignment -Identity <pilotUser> -PolicyType TeamsMeetingPolicy` (or `TEAMSMTG-Validation.ps1 -SampleGroup 'Meeting Scope – Pilot – Group'`).
- **Pass:** each sampled user's **effective** meeting policy is the one their highest-ranked group assignment grants (or Global if unassigned by design).
- **Fail:** a user resolves to the wrong policy → check group membership and assignment ranks (a higher-ranked group can win unexpectedly).

Test 5 — Lobby behaves as configured (live)

- **How:** As a pilot organizer, schedule a meeting; have an **external/anonymous** person and an **internal** person try to join early.
- **Pass:** admission matches `-AutoAdmittedUsers` (e.g. Exec = organizer-only → external waits in lobby; internal per policy). Anonymous join follows the org + policy stance.
- **Fail:** someone bypasses the lobby who shouldn't → recheck the effective policy (Test 4) and the org anonymous switch (Test 2), allow propagation time, retest.

Test 6 — Recording & transcription follow policy (live)

- **How:** In a meeting held under each cohort's policy, attempt **Record** and **Transcribe**.
- **Pass:** recording/transcription are available only where `-AllowCloudRecording` / `-AllowTranscription` are `$true`; unavailable where `$false`.
- **Fail:** a control is available/unavailable against policy → confirm the meeting was held under the expected effective policy and propagation completed.

Test 7 — Presenter role enforced (live)

- **How:** In a meeting, have an **attendee** attempt to share screen / manage participants.
- **Pass:** presenter capability matches `-DesignatedPresenterRoleMode` (e.g. organizer-only mode blocks attendee sharing until promoted).
- **Fail:** attendees can present against policy → recheck effective policy and retest after propagation.

Test 8 — Webinars & town halls governed

- **How:** As a **Standard** cohort user, attempt to create a **webinar** and a **town hall**; repeat as a **Restricted** cohort user.
- **Pass:** creation matches `Events Policy – Standard – Prod` (allowed) vs `Events Policy – Restricted – Prod (blocked). Get-CsTeamsEventsPolicy` shows the intended stance.
- **Fail:** creation allowed/blocked against policy → verify the events-policy assignment for that cohort and propagation.

Test 9 — Meeting template + sensitivity (Teams Premium only)

- **How:** As a **Premium** pilot organizer, schedule using `Meeting Template – Confidential – Prod`; confirm the template appears and its locked options + the `Sensitivity Label – Confidential – Meeting` protections apply. As a **non-Premium** user, confirm the template does not appear.
- **Pass:** Premium organizer sees and can apply the template with its label; non-Premium does not (expected licensing behavior).
- **Fail:** template missing for a Premium user → recheck the template policy publication and Premium licensing (§2.1); do **not** assume it should appear without Premium.

Test 10 — Recording lands in the right place with expiration

- **How:** Record a test meeting under a recording-enabled policy (non-channel and channel).

• **Pass:** non-channel recording is in the organizer's **OneDrive**, channel recording in the team's **SharePoint** site; the file's expiration reflects `-NewMeetingRecordingExpirationDays`.

• **Fail:** recording missing or in the wrong location / no expiration → confirm the organizer's OneDrive is provisioned (§2.5) and the policy's expiration value; coordinate conflicting retention with compliance.

Test 11 — No unintended blast radius

• **How:** Sample **unassigned** users: `Get-CsUserPolicyAssignment -Identity <user> -PolicyType TeamsMeetingPolicy`.

• **Pass:** users not in any assigned cohort resolve to the **Global** baseline you tuned — no surprise policy from a stray assignment.

• **Fail:** an unassigned user has an unexpected policy → find and remove the stray group/user assignment.

Validation report

`TEAMSMTG-Validation.ps1` emits the `Meeting Validation – Teams – Report` (CSV) with a row per check and a PASS/FAIL column. **All rows PASS + the live tests (5–10) confirmed** is the gate to expand to the next wave.

6. Troubleshooting

Common failure modes when configuring Teams meetings, webinars, and policy assignment — each with the likely root cause and the fix. Reuse the reference-model names. Reconnect if needed (`Connect-MicrosoftTeams`; `Connect-MgGraph -Scopes 'Group.Read.All','User.Read.All'`).

A policy change "isn't taking effect" for users

• **Most likely: propagation delay.** New/changed meeting or events policies and group assignments can take minutes (occasionally longer) to reach clients. **Wait and retest**, and have the user fully restart Teams.

• **Or: the user isn't actually assigned the policy you think.** Check the **effective** policy, not the policy definition: `Get-CsUserPolicyAssignment -Identity <user> -PolicyType TeamsMeetingPolicy` (or `TEAMSMTG-Validation.ps1 -SampleGroup ...`). If it resolves to Global, the group assignment didn't apply to them.

• **Or: you edited the wrong policy.** Confirm you changed the policy the user is assigned, not a similarly named one. Names matter — use the exact reference-model names.

A user gets the wrong policy (multiple groups)

- **Root cause:** the user is in **more than one** assigned group, and group assignments resolve by

Rank — the **lowest-Rank** (highest-precedence) assignment for that policy type wins.

- **Fix:** review ranks with `Get-CsGroupPolicyAssignment -PolicyType TeamsMeetingPolicy` (or

`TEAMSMTG-AssignmentReport.ps1`), and set ranks so the intended policy wins. If a user needs an explicit exception, use a direct `Grant-CsTeamsMeetingPolicy -Identity <user>` (per-user overrides group assignment).

Anonymous users still join when you disabled it (or can't when you allowed it)

- **Root cause:** anonymous join is a **two-layer** control. `Set-CsTeamsMeetingConfiguration -Identity

Global -DisableAnonymousJoin `is the **org master switch**; the per-policy` -AllowAnonymousUsersToJoinMeeting` only refines what the org permits.

- **Fix:** confirm the org switch first (`Get-CsTeamsMeetingConfiguration -Identity Global | Select

DisableAnonymousJoin`), then the policy. If the org switch disables anonymous join, no policy can re-enable it. Allow propagation after changing either.

Lobby lets in people it shouldn't (or holds everyone)

- **Root cause:** the effective policy's `-AutoAdmittedUsers` doesn't match intent, or the meeting was

created **before** the policy change and kept its old settings.

- **Fix:** verify the organizer's effective policy value, and test with a **newly scheduled** meeting.

Remember the organizer can also change lobby options **per meeting** (unless a Teams Premium template locks them) — a locked template is the way to enforce it.

Presenter role isn't enforced — attendees can still share

- **Root cause:** `-DesignatedPresenterRoleMode` on the effective policy isn't the restrictive mode you

expect, or the meeting predates the change, or the organizer promoted the attendee to presenter.

- **Fix:** confirm the effective policy mode; test with a fresh meeting; check the meeting's participant

roles. For guaranteed enforcement on sensitive meetings, lock it via a Teams Premium template.

Recording or transcription button is missing (or unexpectedly present)

- **Root cause:** the effective policy's `-AllowCloudRecording` / `-AllowTranscription` value, or a

meeting created before the change.

- **Fix:** verify the effective policy; test with a new meeting. Note both organizer and the recording

initiator's policy can matter — verify the person trying to record has the expected policy.

Recording succeeds but the file is missing / in the wrong place

- **Root cause:** recordings go to **OneDrive** (non-channel meetings, organizer's OneDrive) or

SharePoint (channel meetings, team site) — **not** a separate service. If the organizer has **no OneDrive provisioned** or is **out of quota**, recording can fail or have nowhere to land.

- **Fix:** confirm the organizer has OneDrive licensed/provisioned with quota (§2.5). For channel

meetings, check the team's SharePoint site. Check `-NewMeetingRecordingExpirationDays` and any OneDrive/SharePoint **retention** — the stricter of expiration vs retention governs; coordinate with compliance.

Webinar or town hall creation is blocked (or wrongly allowed)

- **Root cause:** the effective **events policy** (`Events Policy – Standard/Restricted – Prod`) stance

(`-AllowWebinars` / `-AllowTownhalls`) for that user, or the assignment didn't propagate.

- **Fix:** `Get-CsTeamsEventsPolicy` for the definition and `Get-CsUserPolicyAssignment -PolicyType

TeamsEventsPolicy -Identity <user>` for the effective assignment. Confirm the cohort's events-policy group assignment exists and has propagated.

`Set-CsTeamsEventsPolicy` / template cmdlet fails on a parameter

- **Root cause:** events-policy and meeting-template cmdlet **parameters and accepted values are

version-dependent** — a parameter valid in one module/tenant version may not exist in another.

- **Fix:** run `Get-Help Set-CsTeamsEventsPolicy -Full` (and update the **MicrosoftTeams** module),

confirm the exact parameters/values for your version, and adjust. Do **not** guess parameter names — verify in your tenant. For templates, prefer the **Teams admin center** and verify the current PowerShell template cmdlet names.

Meeting template doesn't appear for a user

- **Root cause (most common): no Teams Premium.** Templates, sensitivity labels on meetings, watermark,

and prevent-copy require **Teams Premium** on the **organizer** — without it the template silently does not appear. Or: the template policy isn't published to that user.

- **Fix:** verify Teams Premium licensing (§2.1); verify the template policy is published to the user's

cohort. A **non-Premium** user not seeing the template is **expected**, not a bug.

Sensitivity label can't be attached to the template

- **Root cause:** the label isn't **published** to the relevant users in **Microsoft Purview**, or the

operator lacks the Purview label role.

- **Fix:** confirm with the **compliance owner** that `Sensitivity Label – Confidential – Meeting` (use

the real name) is created and published. **Do not create or redefine the label** — its meaning and protections are the compliance owner's determination.

`Connect-MicrosoftTeams` or Graph fails / wrong cloud

- **Root cause:** wrong cloud environment, missing module, or insufficient role.
- **Fix:** for **Government** clouds connect with the correct environment

(`Connect-MicrosoftTeams -TeamsEnvironmentName TeamsGCCH` / `TeamsDOD` — **verify the value for your cloud**); ensure the **MicrosoftTeams** module is installed/updated (`Install-Module MicrosoftTeams`) and the operator holds **Teams Administrator**. For Graph, connect with `-Scopes 'Group.Read.All','User.Read.All'`.

Rolling back safely

- **Un-assign the group first** — fastest restore of the prior experience:

`Remove-CsGroupPolicyAssignment -GroupId <id> -PolicyType TeamsMeetingPolicy`. Users fall back to their next-ranked assignment or **Global**.

- **Revert a per-user grant:** `Grant-CsTeamsMeetingPolicy -Identity <user> -PolicyName $null`.
- **A custom policy left unassigned affects no one** — you can leave it in place. Only

`Remove-CsTeamsMeetingPolicy` once no assignment references it.

- **Never delete a policy that still has assignments**, and coordinate any label/template change with

the compliance owner.

7. Best Practices

Durable practices for configuring Teams meetings, webinars, and policy assignment. These outlast any single tenant and keep the meeting estate secure and consistent.

Policy design

- **Don't butcher the Global policy — layer custom policies.** Tune **Global (Org-wide default)** to a

safe baseline for everyone unassigned, and express cohort differences in a **small** set of named custom policies (`Standard`, `Exec`, `Restricted`). A sprawl of near-identical policies is unmaintainable.

- **Name every policy** `<Purpose> – <Scope> – <Stage>` **and reuse the name everywhere** — admin center,

scripts, assignments, runbook. Consistent names make audits and troubleshooting fast.

- **Decide the baseline with the business, not in the tool.** Lobby, recording, transcription, and

presenter defaults are security/collaboration decisions (§2.4). The tool applies them; the stakeholders own them.

Security defaults that pay off

- **Close the lobby by default.** Prefer `EveryoneInCompanyExcludingGuests` (or tighter) over `Everyone`; reserve organizer-only lobby for sensitive cohorts. Never leave external/anonymous bypass on by default.
- **Control anonymous at both layers.** Set the org master switch (`DisableAnonymousJoin`) deliberately; the per-policy toggle only refines what the org allows.
- **Default presenter to a restrictive mode** for most cohorts so attendees can't hijack sharing; organizers can still promote presenters per meeting.
- **Keep PSTN lobby-bypass off** unless a specific cohort needs it — dial-in participants are the hardest to identify.

Recording, transcription & storage

- **Treat recording as data.** Set `-NewMeetingRecordingExpirationDays` deliberately and coordinate with the compliance owner's OneDrive/SharePoint retention — the two interact and the stricter one governs.
- **Confirm OneDrive provisioning before enabling recording** for a cohort; a recording with nowhere to land fails silently for the organizer.
- **Don't enable transcription org-wide by reflex** — some jurisdictions and meeting types have consent implications; make it a cohort decision.

Webinars & town halls

- **Govern who can create external events.** Restrict webinar/town-hall creation to the cohorts that need it via the events policy; don't leave it open to the whole org.
- **Prefer town halls over legacy live events** for new large-broadcast needs — Microsoft is steering live events toward town halls; verify the retirement timeline for your cloud before building new live events.

Teams Premium (templates & sensitivity)

- **Verify Premium entitlement before designing templates.** If the cohort has no Teams Premium, templates/sensitivity/watermark silently do nothing — scope them to Premium users or defer.
- **Attach labels, don't author them.** The sensitivity label and its protections are the **compliance

owner's** determination in Purview; the template only attaches an existing published label.

• **Use templates to lock the sensitive experience** (forced lobby, no recording, restricted presenting) so organizers can't accidentally loosen it.

Assignment & change discipline

• **Assign by group, not per user.** Group assignment keeps the experience aligned to identity/HR reality and survives joiners/movers/leavers. Reserve per-user grants for exceptions.

• **Rank assignments deliberately.** A user in multiple assigned groups gets the **lowest-Rank** (highest precedence) assignment per policy type — document the rank scheme and test effective policy (Test 4).

• **Always pilot first.** Assign to `Meeting Scope – Pilot – Group`, validate against §5, and only then expand in waves. Meeting changes touch everyone's daily experience.

• **Keep the rollback ready.** Roll back **assignments first** (fastest restore), then policies; never delete a policy that still has assignments (§3.11).

• **Re-run discovery periodically.** `TEAMSMTG-Discovery.ps1` on a schedule catches drift — new policies, stray assignments, or a Global baseline that changed under you.

Cloud & versioning

• **Confirm Commercial vs Government early** and use the matching URLs/endpoints and PowerShell environment throughout; some events/Premium features differ by cloud.

• **Treat version-dependent parameters as "verify in your tenant."** Events-policy and template cmdlet parameters evolve; check the accepted values for your module version before scripting them broadly.

• **Automate safe-by-default.** Keep the scripts read-only / `-WhatIf` unless `-Commit` is explicit, and never store credentials — always modern-auth `Connect-*`.

8. Deliverables Summary

Everything included in this solution, on one page.

Documentation

Document	What it gives you
1. Solution Overview	What meeting/webinar policy configuration is here, how it works, the reference model/artifacts, benefits, limitations, and the explicit boundary vs Teams Phone, Purview label definition, and other Teams solutions.

Document	What it gives you
2. Requirements	Solution-specific prerequisites (Teams Premium gate for templates/sensitivity, least-privilege roles, cohort groups, agreed baseline, recording storage, compliance-defined labels, Commercial-vs-Gov) with verify/implement steps — on top of the shared Teams requirements.
3. Implementation	Exact ordered steps: assess → design the policy set → org settings → build meeting policies → webinars/town halls → templates + sensitivity (Premium) → pilot assign by group → validate → expand in waves, with pass/fail gates and a rollback. Every artifact named.
5. Testing & Validation	Eleven tests (policies exist, org anonymous stance, assignments/ranks, effective policy, lobby, recording/transcription, presenter role, webinars/town halls, template+sensitivity, recording storage, no blast radius), each with pass/fail.
6. Troubleshooting	Common failure modes (policy not taking effect, wrong effective policy, anonymous still joining, template not showing, recording missing, webinar blocked) with root cause and fix.
7. Best Practices	Durable practices for policy design, security defaults, recording/storage, webinars, Teams Premium, and assignment/change discipline.
Executive Summary	Crisp outcome/approach/why-low-risk summary for decision-makers.

PowerShell scripts (5) — MicrosoftTeams module / Microsoft Graph

All are **safe by default** (read-only or `-WhatIf`/report unless an explicit `-Commit`), use modern-auth `Connect-MicrosoftTeams` / `Connect-MgGraph` (no stored credentials), and reuse the artifact names from the reference model.

Script	What it does	Safe default
TEAMSMTG-Discovery.ps1	Read-only assessment: every meeting policy, the events policy, org meeting configuration, recording/storage-relevant settings, and existing group assignments. Emits `Meeting Posture – Teams – Discovery`.	Read-only

Script	What it does	Safe default
TEAMSMTG-Set-MeetingPolicy.ps1	Idempotently creates/updates the meeting policies (`Standard`/`Exec`/`Restricted` + pilot, optionally Global) with lobby, anonymous, recording, transcription, presenter-role, and recording-expiration settings.	Preview unless `-Commit`; `-WhatIf`
TEAMSMTG-Set-EventsAndTemplates.ps1	Idempotently creates/updates the events policy (`Events Policy – Standard/Restricted – Prod`) for webinars + town halls, and reports the Teams Premium template/sensitivity readiness (flags what must be done in the admin center + Purview).	Preview unless `-Commit`; `-WhatIf`
TEAMSMTG-Validation.ps1	Read-only pass/fail validation of policy existence, org anonymous stance, assignments/ranks, and per-user effective policy for a sample group; emits `Meeting Validation – Teams – Report`.	Read-only
TEAMSMTG-AssignmentReport.ps1	Scope/health helper: reports group policy assignments and the effective policy per sampled user; with `-AssignGroup … -Commit`, applies a ranked `New-CsGroupPolicyAssignment` (previews with `-WhatIf`).	Report unless `-Commit`; `-WhatIf`

Licensing note: *meeting* ***templates***, ***sensitivity labels on meetings***, ***watermark***, *and* ***prevent-copy*** *are* ***Teams Premium*** *— verify entitlement in your tenant/plan before relying on them.* ***Sensitivity-label meaning and retention are a Microsoft Purview / compliance determination****, not an implementer decision — the template only attaches an existing published label.*

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

TEAMSMTG-AssignmentReport.ps1

TEAMSMTG-Discovery.ps1

TEAMSMTG-Set-EventsAndTemplates.ps1

TEAMSMTG-Set-MeetingPolicy.ps1

TEAMSMTG-Validation.ps1